# BEE 4750 Lab 1: Julia and GitHub Basics

**Name**:

**ID**:

> **Due Date**
>
> Thursday, 8/28/25, 9:00pm

## Setup

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
  Activating project at `c:\Users\maozb\Documents\Cornell\BEE4750\Lab1\lab1-maozbizan`
  Installing known registries into `C:\Users\maozb\.julia`
       Added `General` registry to C:\Users\maozb\.julia\registries
   Installed GR_jll               v0.73.17+0
   Installed libdecor_jll            v0.2.2+0
   Installed JpegTurbo_jll           v3.1.1+0
   Installed libfdk_aac_jll          v2.0.4+0
   Installed LERC_jll              v4.0.1+0
   Installed Libmount_jll            v2.41.0+0
   Installed x265_jll              v4.1.0+0
   Installed Preferences             v1.4.3
   Installed LoggingExtras           v1.1.0
   Installed Opus_jll              v1.5.2+0
   Installed Xorg_xkbcomp_jll          v1.4.7+0
```

```
Installed RelocatableFolders        v1.0.1
Installed Measures              v0.3.2
Installed Unitful              v1.24.0
Installed ConcurrentUtilities       v2.5.0
Installed Contour              v0.6.3
Installed Xorg_xcb_util_wm_jll       v0.4.2+0
Installed Grisu               v1.0.2
Installed PlotUtils             v1.4.3
Installed RecipesPipeline          v0.6.12
Installed Xorg_xcb_util_image_jll     v0.4.1+0
Installed OpenSSL              v1.5.0
Installed Xorg_libSM_jll          v1.2.6+0
Installed libsodium_jll           v1.0.21+0
Installed Xorg_xcb_util_jll         v0.4.1+0
Installed Cairo_jll             v1.18.5+0
Installed DelimitedFiles          v1.9.1
Installed HTTP                v1.10.17
Installed Fontconfig_jll          v2.16.0+0
Installed Statistics             v1.11.1
Installed Xorg_libxkbfile_jll        v1.1.3+0
Installed EpollShim_jll           v0.0.20230411+1
Installed Xorg_libXinerama_jll       v1.1.6+0
Installed Xorg_libXau_jll          v1.0.13+0
Installed FFMPEG              v0.4.4
Installed Missings             v1.2.0
Installed ColorSchemes           v3.30.0
Installed Showoff              v1.0.3
Installed IrrationalConstants       v0.2.4
Installed JSON                v0.21.4
Installed Pango_jll             v1.56.3+0
Installed xkbcommon_jll           v1.9.2+0
Installed SimpleBufferStream        v1.2.0
Installed XZ_jll              v5.8.1+0
Installed Xorg_xcb_util_keysyms_jll    v0.4.1+0
Installed PtrArrays             v1.3.0
Installed Bzip2_jll             v1.0.9+0
Installed IJulia              v1.29.2
Installed HarfBuzz_jll           v8.5.1+0
Installed PlotThemes            v3.3.0
Installed NaNMath              v1.1.3
Installed LZO_jll              v2.10.3+0
Installed fzf_jll              v0.61.1+0
Installed SoftGlobalScope          v1.1.0
```

```
Installed TranscodingStreams        v0.11.3
Installed FriBidi_jll            v1.0.17+0
Installed UnicodeFun             v0.4.1
Installed ZeroMQ_jll             v4.3.6+0
Installed GLFW_jll               v3.4.0+2
Installed MbedTLS                v1.1.9
Installed x264_jll               v10164.0.1+0
Installed Colors                 v0.13.1
Installed DataStructures           v0.18.22
Installed FreeType2_jll            v2.13.4+0
Installed Compat                 v4.18.0
Installed StatsAPI               v1.7.1
Installed CodecZlib              v0.7.8
Installed JLFzf                  v0.1.11
Installed Xorg_libxcb_jll          v1.17.1+0
Installed StatsBase              v0.34.5
Installed libpng_jll             v1.6.50+0
Installed mtdev_jll              v1.1.7+0
Installed ExceptionUnwrapping        v0.1.11
Installed libaom_jll             v3.12.1+0
Installed Scratch                v1.3.0
Installed ColorTypes             v0.12.1
Installed Dbus_jll               v1.16.2+0
Installed eudev_jll              v3.2.14+0
Installed Xorg_libXext_jll          v1.3.7+0
Installed Conda                  v1.10.2
Installed TensorCore             v0.1.1
Installed Zstd_jll               v1.5.7+1
Installed Xorg_xcb_util_cursor_jll     v0.1.5+0
Installed Expat_jll              v2.6.5+0
Installed Libtiff_jll             v4.7.1+0
Installed ZMQ                  v1.4.1
Installed Parsers                v2.8.3
Installed Plots                 v1.40.17
Installed Format                 v1.3.7
Installed JLLWrappers             v1.7.1
Installed Xorg_libXrender_jll        v0.9.12+0
Installed Libffi_jll             v3.4.7+0
Installed ColorVectorSpace          v0.11.0
Installed OrderedCollections         v1.8.1
Installed libevdev_jll            v1.13.4+0
Installed libinput_jll            v1.28.1+0
Installed Ogg_jll                v1.3.6+0
```

```
Installed Xorg_libXi_jll          v1.8.3+0
Installed Vulkan_Loader_jll        v1.3.243+0
Installed Qt6ShaderTools_jll       v6.8.2+1
Installed Reexport              v1.2.2
Installed LogExpFunctions          v0.3.29
Installed Qt6Declarative_jll       v6.8.2+1
Installed Xorg_libXcursor_jll      v1.2.4+0
Installed MacroTools            v0.5.16
Installed Xorg_libICE_jll          v1.1.2+0
Installed Libuuid_jll           v2.41.0+0
Installed Xorg_xcb_util_renderutil_jll  v0.3.10+0
Installed AliasTables           v1.1.3
Installed DocStringExtensions       v0.9.5
Installed StableRNGs            v1.0.3
Installed Pixman_jll            v0.44.2+0
Installed libass_jll            v0.17.4+0
Installed Graphite2_jll          v1.3.15+0
Installed Wayland_jll           v1.24.0+0
Installed Xorg_xtrans_jll          v1.6.0+0
Installed Latexify              v0.16.8
Installed Xorg_xkeyboard_config_jll    v2.44.0+0
Installed BitFlags              v0.1.9
Installed FFMPEG_jll            v7.1.1+0
Installed LLVMOpenMP_jll          v18.1.8+0
Installed DataAPI              v1.16.0
Installed OpenSSL_jll            v3.5.1+0
Installed FixedPointNumbers        v0.8.5
Installed Xorg_libXrandr_jll        v1.5.5+0
Installed RecipesBase           v1.3.4
Installed LAME_jll             v3.100.3+0
Installed Qt6Wayland_jll          v6.8.2+1
Installed Xorg_libXfixes_jll        v6.0.1+0
Installed PrecompileTools         v1.2.1
Installed GettextRuntime_jll        v0.22.4+0
Installed Libiconv_jll           v1.18.0+0
Installed Qt6Base_jll            v6.8.2+1
Installed LaTeXStrings           v1.4.0
Installed URIs                v1.6.1
Installed libvorbis_jll           v1.3.8+0
Installed VersionParsing          v1.3.0
Installed Glib_jll             v2.84.3+0
Installed Libglvnd_jll           v1.7.1+1
Installed Requires             v1.3.1
```

```
   Installed Xorg_libXdmcp_jll        v1.1.6+0
   Installed Xorg_libX11_jll         v1.8.12+0
   Installed Unzip               v0.2.0
   Installed UnitfulLatexify          v1.7.0
   Installed SortingAlgorithms        v1.2.1
   Installed GR                v0.73.17
    Building Conda → `C:\Users\maozb\.julia\scratchspaces\44cfe95a-1eb2-52ea-b672-e2afdf69b7
    Building IJulia → `C:\Users\maozb\.julia\scratchspaces\44cfe95a-1eb2-52ea-b672-e2afdf69b
Precompiling project...
   1008.5 ms   LaTeXStrings
   1024.0 ms   StatsAPI
   1019.0 ms   Contour
   1053.1 ms   TensorCore
   1075.9 ms   Measures
   1304.2 ms   VersionParsing
   1043.4 ms   Statistics
   2043.0 ms   Format
   1210.7 ms   OrderedCollections
   1150.5 ms   Requires
   1014.6 ms   StableRNGs
   1582.1 ms   Grisu
    819.4 ms   Reexport
   1160.5 ms   Unzip
   1167.9 ms   DocStringExtensions
    826.9 ms   SimpleBufferStream
    865.0 ms   PtrArrays
   1387.1 ms   URIs
   1172.4 ms   TranscodingStreams
   2197.7 ms   IrrationalConstants
   1292.3 ms   NaNMath
   1278.1 ms   DelimitedFiles
    977.2 ms   DataAPI
   1010.3 ms   BitFlags
   3864.5 ms   MacroTools
    991.3 ms   Scratch
   1548.9 ms   ConcurrentUtilities
   1095.2 ms   LoggingExtras
   1166.1 ms   Compat
   1396.9 ms   Preferences
   2067.2 ms   MbedTLS
   1397.6 ms   ExceptionUnwrapping
    949.6 ms   Showoff
   1837.8 ms   SoftGlobalScope
```

```
1024.1 ms    AliasTables
2212.9 ms    Statistics → SparseArraysExt
1059.7 ms    CodecZlib
3037.2 ms    UnicodeFun
1343.0 ms    LogExpFunctions
1026.0 ms    Missings
1124.9 ms    RelocatableFolders
1021.7 ms    Compat → CompatLinearAlgebraExt
3830.0 ms    FixedPointNumbers
1152.6 ms    JLLWrappers
1043.7 ms    PrecompileTools
1120.2 ms    Graphite2_jll
1146.6 ms    OpenSSL_jll
 973.5 ms    Libmount_jll
 989.0 ms    EpollShim_jll
2523.8 ms    ColorTypes
1116.1 ms    LLVMOpenMP_jll
1125.7 ms    Bzip2_jll
2796.4 ms    DataStructures
5071.8 ms    Latexify
1272.6 ms    libsodium_jll
1171.9 ms    Xorg_libXau_jll
1271.8 ms    Xorg_libICE_jll
1327.7 ms    libpng_jll
1317.1 ms    libfdk_aac_jll
1158.3 ms    LERC_jll
1174.3 ms    LAME_jll
1182.6 ms    fzf_jll
1303.8 ms    JpegTurbo_jll
1198.4 ms    Ogg_jll
1342.5 ms    XZ_jll
1171.4 ms    mtdev_jll
1126.4 ms    Xorg_libXdmcp_jll
1201.4 ms    x265_jll
1226.2 ms    x264_jll
1228.7 ms    Zstd_jll
1278.0 ms    libaom_jll
1234.6 ms    LZO_jll
1258.1 ms    Expat_jll
1294.8 ms    Opus_jll
1144.5 ms    Xorg_xtrans_jll
1060.2 ms    libevdev_jll
1230.7 ms    Libiconv_jll
```

```
 1120.9 ms    eudev_jll
 1254.6 ms    Libffi_jll
 1113.5 ms    Libuuid_jll
 1240.6 ms    FriBidi_jll
 1431.9 ms    ColorTypes → StyledStringsExt
 2857.5 ms    RecipesBase
 2681.6 ms    OpenSSL
 1307.2 ms    Pixman_jll
 1371.1 ms    FreeType2_jll
 3231.8 ms    ColorVectorSpace
 1222.2 ms    SortingAlgorithms
 1292.6 ms    Latexify → SparseArraysExt
  945.8 ms    Xorg_libSM_jll
 1157.7 ms    ZeroMQ_jll
  969.4 ms    Xorg_libxcb_jll
 1224.9 ms    libvorbis_jll
 1299.6 ms    Libtiff_jll
 2038.3 ms    JLFzf
 1013.0 ms    Dbus_jll
 1475.7 ms    GettextRuntime_jll
 1193.8 ms    libinput_jll
 1217.3 ms    Wayland_jll
 6905.3 ms    Colors
  867.7 ms    Xorg_xcb_util_jll
 1415.2 ms    Fontconfig_jll
  958.1 ms    Xorg_libX11_jll
 1503.1 ms    Glib_jll
 3607.1 ms    StatsBase
 3554.1 ms    ZMQ
 1067.1 ms    Xorg_xcb_util_image_jll
 1177.2 ms    Xorg_xcb_util_keysyms_jll
 1090.8 ms    Xorg_xcb_util_renderutil_jll
 1161.1 ms    Xorg_xcb_util_wm_jll
 3067.4 ms    Xorg_libXext_jll
 2871.1 ms    Xorg_libXfixes_jll
 3087.9 ms    Xorg_libXrender_jll
 1421.8 ms    Xorg_libXinerama_jll
30167.2 ms    Unitful
 1654.9 ms    Xorg_libxkbfile_jll
 1555.7 ms    Xorg_xcb_util_cursor_jll
 7622.5 ms    ColorSchemes
17624.4 ms    Parsers
 1161.5 ms    Libglvnd_jll
```

```
 1151.7 ms    Xorg_libXi_jll
 1169.3 ms    Xorg_libXrandr_jll
 1141.4 ms    Xorg_libXcursor_jll
 1580.8 ms    Unitful → PrintfExt
 1885.6 ms    Cairo_jll
  964.6 ms    Xorg_xkbcomp_jll
 1102.1 ms    Xorg_xkeyboard_config_jll
 1551.5 ms    HarfBuzz_jll
 2614.4 ms    JSON
 1276.0 ms    xkbcommon_jll
 2690.0 ms    UnitfulLatexify
 1787.0 ms    libass_jll
 1754.9 ms    Pango_jll
 1346.1 ms    Vulkan_Loader_jll
 2048.0 ms    Conda
 1120.4 ms    libdecor_jll
 1215.7 ms    GLFW_jll
 2352.8 ms    FFMPEG_jll
 2485.9 ms    Qt6Base_jll
 1410.1 ms    FFMPEG
 1469.4 ms    Qt6ShaderTools_jll
 1798.2 ms    GR_jll
 6969.2 ms    IJulia
 3742.0 ms    Qt6Declarative_jll
 1621.4 ms    Qt6Wayland_jll
15146.3 ms    PlotUtils
26817.2 ms    HTTP
 3695.2 ms    PlotThemes
 5055.7 ms    GR
 5464.0 ms    RecipesPipeline
65006.7 ms    Plots
 5232.0 ms    Plots → UnitfulExt
 6028.3 ms    Plots → IJuliaExt
153 dependencies successfully precompiled in 130 seconds. 35 already precompiled.
```

This next cell loads packages which are required for the rest of the code evaluation. In this case, we only need to load the `Plots.jl` plotting package, but you will see others over the course of the semester (and can add more if desired; just make sure that you've added the new packages to the environment). Standard Julia practice is to load all of the needed packages at the top of the file.

**Warning**

Loading packages can take a while, especially the first time! Julia tries to precompile all of the packages you're using so repeat use is faster, but this can be quite slow at first.

```
using Plots
```

## Introduction

### Julia

Julia is an up-and-coming language, originally developed for scientific programming. While learning a new programming language always has its hiccups, the good news is that if you've programmed in a high-level language such as Python or MATLAB, most Julia concepts should look familiar.

If you have not successfully set up Julia, follow the instructions in Tools Setup and/or ask for help.

You can use other editors for this course, but our recommendation is Visual Studio Code with the Julia extension, which will make life a *lot* simpler! You should have set this up by following the Tools Setup instructions, but if not, do so now and/or ask for help.

### Jupyter Notebooks

Jupyter notebooks integrate text and equations in Markdown with Julia (or Python, or R) code. To do this, Jupyter notebooks consist of two types of "cells": code cells and Markdown (text) cells.

Click once on this section of text. A box will appear around this text (and some areas above/below it) - all of that is within this cell.

Markdown is a text markup framework for formatting language that makes things look pretty when viewed across different platforms: web browsers, notebooks, and so forth. Text written in Markdown can also include hyperlinks, LaTeX equations, section headers, and images, among other features (most of the course website and the lecture notes were all written in Markdown!). Here is a basic Markdown cheat sheet.

What you are looking at right now is the formatted text after the Markdown is processed. To see the raw Markdown, do one of:

- press `Enter` while that cell is selected, or
- double-click on that cell.

---

A couple of the features you will see in this Markdown cell:

- The `---` command creates a horizontal line. This is also nice for separating sections.
- Backticks (`` `...` ``) can be used to format and highlight code, keystrokes, etc.
- The `#` sign is used to create a new section header; two `#` signs (`##`) is used to create a new subsection header; `###` creates a subsubsection, and so on.
- You can create a bulleted list by using the asterisk `*` or a dash `-` and a space.
- You can create regular text by just typing as usual.
- You can create **bold-faced text** by wrapping it with two asterisks on both sides.
- You can create *italicized text* by wrapping it with a single asterisk on both sides.
- To create a new paragraph, you must include a blank line between the old and new paragraphs.

At this point you might be wondering how to turn this cell back into the fully formatted Markdown text instead of the raw Markdown you're probably still looking at. You have a couple of options, depending on your platform, but the most consistent is to type `Shift + Enter` to **execute** the cell (this is also how to run code, but more on that later).

Additionally, you will frequently need to create new cells in your Jupyter notebooks. How you do this will depend on how you interact with the notebook, but try to figure this out now.

One tip is to think carefully about what bits of code should be in the same cell, as you typically only see output from the last command in a cell. For example, compare the following:

```
x = 5
sin(x)
```

```
-0.9589242746631385
```

with

```
x = 5
```

```
5
```

```
sin(x)
```

```
-0.9589242746631385
```

In Julia, you can also suppress the output of a command with a semi-colon:

```
sin(x);
```

which can help if you want to split some code out for clarity or to insert some text prior to it, but don't want to clutter the notebook with its output.

For code cells, to execute the commands within the cell, we also press `Shift+Enter`.

Finally, **make sure that you evaluate all of the code cells in order before submitting**. One bad outcome with notebooks occurs when cells are evaluated out of order, so fixed bugs and edits in previous cells do not get a chance to propagate down. You can do this with the `Run All` command in whichever interface you're using to edit your notebook.

### Julia Basics

There are many tutorials and references for Julia, including a basics overview on the class website. Please feel free to reference these as you work through any part of the course.

### Formatting Math

It will often be helpful to include nicely-formatted mathematics in a notebook. Markdown accomodates this using LaTeX syntax. A LaTeX cheatsheet is available on the class website, and many other guides exist online.

Below is an example of a formatted equation:

$$x = 5.$$

### Looking For Help

There is no shame in using Google, or other resources, for help when programming. There are many, many times when you can't quite get the syntax to work, can't quite figure out the right package or command to use, or are feeling too lazy or overwhelmed (I'm not judging either of those!) to dig through the documentation. Some good resources include:

- Stack Overflow is a treasure trove of answers;
- The official Julia forum and the Julia Subreddit are also very useful.

You are also highly encouraged to post on Ed Discussion, though getting a response might be less immediate. Just be mindful that to get good answers, you have to help people help you, and **make sure to give credit to any resources that were helpful**!

## Exercises (3 points)

Use your understanding of Julia syntax and the GitHub workflow to complete the following (hopefully short) exercises. Convert your completed lab assignment to a PDF and submit it to the Gradescope Assignment "Lab 1".

**Remember to**:

- Include a (succint but clear) writeup of the core idea underlying your code, through some combination of equations, text, and algorithms. As you are not required to submit your code, we will not be looking at it in detail, and instead will rely on those writeups to assess whether your approach is correct.
- If using the notebook, evaluate all of your code cells, in order (using a `Run All` command). This will make sure all output is visible and that the code cells were evaluated in the correct order.
- Tag each of the problems when you submit to Gradescope; a 10% penalty will be deducted if this is not done.

### Computing a Dot Product

Given two numeric arrays `x` and `y`, write a function to compute their dot product if they have equal length, and return an error if not (this is useful for debugging!). Use the following code as a starting point. **Do not use a built-in function**.

```
function dot_product(x, y)
    if length(x) == length(y) # insert test condition for equal lengths
        dp = 0
        for i=1:length(x)
            dp += x[i]*y[i]
        end
        return(dp)
    else
        throw(DimensionMismatch("length of x not equal to length of y"))
    end
end
```

```
dot_product (generic function with 1 method)
```

Here are some tests to make sure your code works as intended. Tests like these are useful to make sure everything works as intended. One reason to split your code up into functions is that it makes it straightforward to write tests to make sure each piece of your code works, which makes it easier to identify where errors are occuring.

```
dot_product([1 2 3], [4 5 6])
```

```
32
```

If you know the value you should get, you can write a more formal test using the `@assert` macro, which is a good way to "automate" checking (since you get an obvious error if the code doesn't work as desired):

```
@assert dot_product([1 2 3], [4 5 6]) == 32
```

Let's also make sure we get an error when the dimensions of the two vectors don't match:

```
dot_product([1 2 3], [4 5])
```

```
DimensionMismatch: DimensionMismatch: length of x not equal to length of y
DimensionMismatch: length of x not equal to length of y



Stacktrace:

 [1] dot_product(x::Matrix{Int64}, y::Matrix{Int64})

   @ Main c:\Users\maozb\Documents\Cornell\BEE4750\Lab1\lab1-maozbizan\jl_notebook_cell_df34

 [2] top-level scope

   @ c:\Users\maozb\Documents\Cornell\BEE4750\Lab1\lab1-maozbizan\jl_notebook_cell_df34fa98e
```

**Problem 1 write-up (computing a dot product):**

I used the skeleton of the code that was already written in the assignment. First, I used an "if" statement to make sure that the two arrays were of the same length. If the arrays are not of a different length, the code throws an error with the message that the arrays are not of the same length.

If the arrays are of the same length, then the code creates a new variable with a value of zero (dp); the products of each corresponding entry in the arrays will be added to this variable. I wrote a for-loop using the length of the arrays as the number of iterations. For each position in the arrays, the for loop multiplies the i-th value from array x with the i-th value from array

y, and adds this to the variable created earlier (dp). Then, once the for-loop is done, the function returns the accumulated value of the variable dp, which is the dot product.

The answer that I got for the example arrays is 32, which aligns with expectations, and I also received the correct error message for the example where arrays were a different length.

### Making a Plot

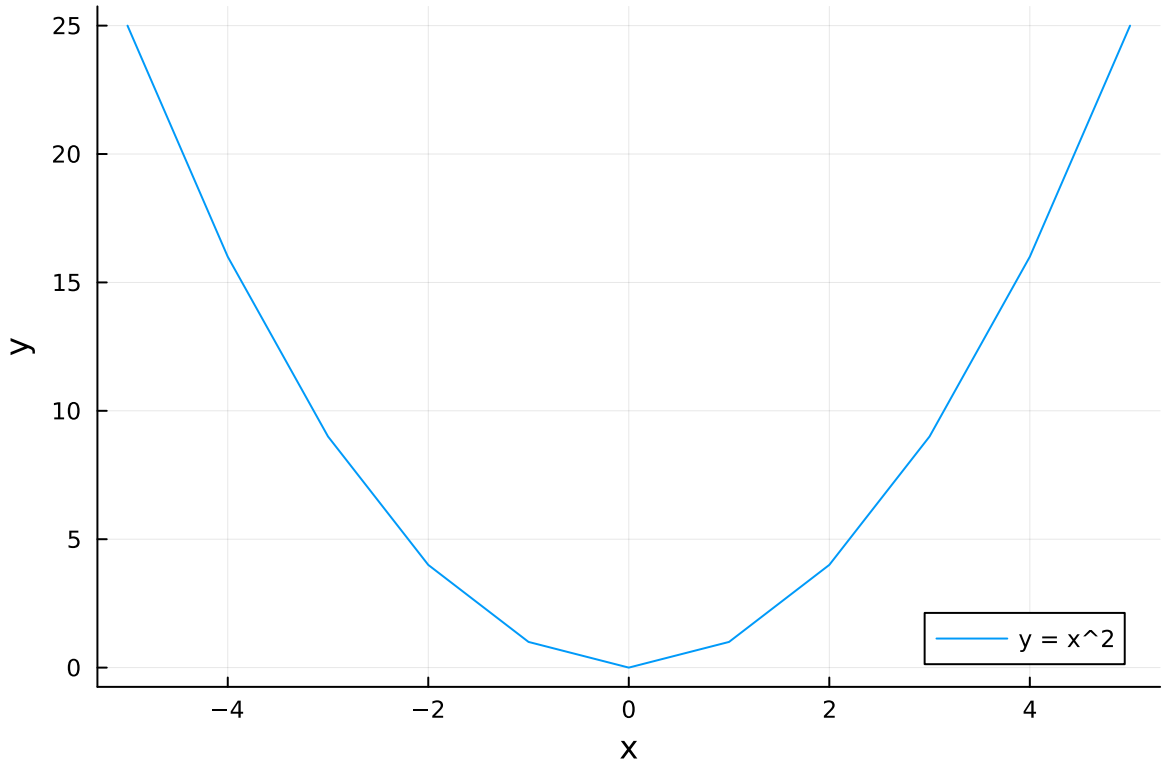
Write a function to compute the square of an integer `x`. Evaluate this function for integers between $x = -5$ and $x = 5$ and make a plot of the squared values (you can find a quick guide to making various types of plots here). Make sure to label your axes.

```
# insert your code here
function square_plot(x)
    square = x*x
    return square
end
```

```
square_plot (generic function with 1 method)
```

```
vals = collect(-5:5)
squares = []
for value in vals
    push!(squares,square_plot(value))
end
```

```
plot(vals,squares, label="y = x^2")
xlabel!("x")
ylabel!("y")
```




**Problem 2 write-up (making a plot)**

First, I defined a function that takes a value x as input and returns the value of x squared.

Then, I created an array with every integer value from -5 to 5, for a total of 11 values. I then created a new empty array to eventually contain the squared values. Next, I wrote a for-loop to square each value in the -5 to 5 array and append each value to the new array. Finally, I plotted the two arrays, with the values on the x-axis and the squares on the y-axis, and added a legend and axis labels.

## Commit and Push Your Changes to GitHub

After completing the previous two exercises, commit your solution file (notebook or otherwise) and push to GitHub. Use an informative commit message which makes it clear what changes you've made. The specific workflow for this will vary depending on how you're writing up your solutions; please search for specifics and ask for help as needed!

### Useful Commit Sizes

Ideally, you'd commit whenever you make a "substantial" enough change that you want to lock in, such as writing the core code for a problem or completing a problem, if you're preparing code to be used elsewhere (by yourself or others), or if you want to ask for help. `git` lets you revert changes back to a previous commit, so it's easy to undo changes or updates which broke something that was previously working, so changing too many things at once can make it hard to keep track of what worked when.

But in this case, go ahead and just commit after finishing the problems.

Push the repository with these commits to GitHub and take a screenshot of the repository page (`https://github.com/BEE4750-FA25/<username>/lab01`) which shows the updated repository. Include that screenshot in your submission as the solution to this problem.

## Submitting PDF

**Important**

These submission instructions will not be repeated on future assignments!

Export your writeup as a PDF (this doesn't have to be a notebook if you're struggling to export: you can do your writeup in *e.g.* Microsoft Word) and submit it to the "Lab 1" assignment on Gradescope. **Make sure that you tag pages corresponding to relevant problems to avoid a 10% penalty**.

**Printing Code to PDF**

You are not required to submit your code when submitting assignments. However, when printing a notebook to PDF (whether you do this locally or via the GitHub Action), long lines will run off the edge of code cells, which may result in comments or code being hidden. If you see this, go back to the notebook and break up long lines into shorter onces (for example, see the comment in the above code cell) to ensure key parts of your results aren't missing.